Jeu de contrôle fictif (L0) — description et résultats attendus

Projet P05 — Analyse distribuée des avis clients et des tendances ISIMa — Mastère 1 IA & DS — Big Data — 2026/2027 **Groupe :** ..... · **Fichier décrit :** `data/toy/toy_reviews.jsonl` (et sa version compressée `.jsonl.gz`)

---

1. Objectif

Les trois jobs MapReduce (MR1, MR2, MR3) doivent produire des résultats **exacts**, vérifiables par un calcul manuel. Cela est impossible sur 2,1 millions d'avis. Le jeu L0 est donc un petit corpus **entièrement fictif**, écrit à la main, dont les résultats attendus sont connus à l'avance. Il répond aux tests T01 à T05 du cahier des charges et au « tableau des sorties attendues » exigé en §17.

Aucune donnée réelle n'est utilisée. Tous les identifiants sont préfixés `TOY_` pour que le jeu soit reconnaissable comme un test.

2. Format

Le fichier est au format **JSON Lines** : un enregistrement JSON par ligne, comme le fichier réel `Appliances.jsonl.gz`. Il contient **15 lignes** :

- 12 avis valides, répartis sur 3 produits fictifs ;
- 3 lignes **volontairement invalides** (test T01, voir §6).

Dictionnaire des champs

Les champs sont ceux du jeu réel Amazon Reviews'23 (le champ `images`, non utilisé, est omis).

Champ	Type	Signification	Exemple
`rating`	nombre	Note de 1 à 5 étoiles	`5.0`
`title`	texte	Titre de l'avis	`"Quiet and reliable"`
`text`	texte	Contenu de l'avis ; source des tokens pour MR1/MR2	`"Very quiet dishwasher..."`
`asin`	texte	Identifiant d'une **variante** du produit	`TOY_A1`
`parent_asin`	texte	Identifiant du **produit** regroupant ses variantes ; **clé de travail** (le « document » du TF-IDF)	`TOY_A`
`user_id`	texte	Identifiant (fictif ici) de l'auteur ; jamais affiché dans les résultats	`TOY_U1`
`timestamp`	entier	Date de l'avis en **millisecondes** Unix (UTC)	`1672574400000` = 2023-01-01 12:00 UTC
`helpful_vote`	entier	Nombre de votes « utile »	`2`
`verified_purchase`	booléen	Achat vérifié ou non	`true`

Remarque : `TOY_A` possède deux variantes (`TOY_A1` , `TOY_A2`) mais un seul `parent_asin` . Les calculs se font toujours par `parent_asin` .

3. Contenu : 3 produits, 12 avis

Produit (`parent_asin`)	Avis	Mois couverts	Notes
`TOY_A`	4	2023-01, 2023-02, 2023-03 (×2)	5, 4, 2, 3
`TOY_B`	3	2023-01 (×2), 2023-02	1, 1, 5
`TOY_C`	5	2023-01, 2023-02, 2023-03 (×2), 2023-04	4, 5, 4, 3, 5

4. Cas de test couverts volontairement

Cas	Comment il est couvert	Job / test
Mot présent dans **tous** les produits	`delivery` apparaît dans A, B et C : df = N, donc IDF = 0	MR2 / T04
Mot partagé par une partie des produits	`quiet` (A et C), `filter` (A et B)	MR2 / T04
Mot **distinctif** d'un seul produit	`broke` (B), `reliable` (A)	MR2 / T04
Mots répétés dans un produit	plusieurs occurrences de `quiet`, `broke`, `easy`…	MR1 / T03
Négation et ponctuation	« Not noisy », « Not reliable », points, virgules, point d'exclamation	T02
Plusieurs avis dans le même mois	A en 2023-03 (2 avis), B en 2023-01 (2 avis), C en 2023-03 (2 avis)	MR3 / T05
Notes des trois classes (1–2 / 3 / 4–5)	présentes dans le corpus	MR3 / T05
Variantes d'un même produit	`TOY_A1` / `TOY_A2` sous `TOY_A`	clé `parent_asin`

5. Résultats attendus

5.1 MR3 — profils mensuels (calculés à la main)

`classes` = (low 1–2 ★, neutre 3 ★, high 4–5 ★).

Produit	Mois	Effectif	Somme des notes	Moyenne	low / neutre / high
TOY_A	2023-01	1	5	5,0	0 / 0 / 1
TOY_A	2023-02	1	4	4,0	0 / 0 / 1
TOY_A	2023-03	2	5	2,5	1 / 1 / 0
TOY_B	2023-01	2	2	1,0	2 / 0 / 0
TOY_B	2023-02	1	5	5,0	0 / 0 / 1
TOY_C	2023-01	1	4	4,0	0 / 0 / 1
TOY_C	2023-02	1	5	5,0	0 / 0 / 1
TOY_C	2023-03	2	7	3,5	0 / 1 / 1
TOY_C	2023-04	1	5	5,0	0 / 0 / 1

Contrôles : 9 lignes de sortie ; la somme des effectifs vaut 12 (= nombre d'avis valides) ; pour chaque ligne, low + neutre + high = effectif ; toutes les moyennes sont dans [1, 5].

5.2 MR2 — fréquence documentaire et IDF (exemples)

Avec N = 3 produits et la formule `idf = ln((N+1)/(df+1))` :

Terme	Produits contenant le terme	df	idf
`delivery`	A, B, C	3	ln(4/4) = **0**
`quiet`	A, C	2	ln(4/3) ≈ **0,2877**
`filter`	A, B	2	ln(4/3) ≈ **0,2877**
`broke`	B	1	ln(4/2) ≈ **0,6931**
`reliable`	A	1	ln(4/2) ≈ **0,6931**

Un mot présent partout (`delivery`) a donc un poids nul, tandis qu'un mot propre à un produit (`broke`) a le poids le plus élevé. Les valeurs complètes de `tf_raw` et de `tfidf` par couple (produit, terme) dépendent des règles NLP finales (tokenisation, stop words) ; elles seront publiées dans le tableau des sorties attendues lors des Semaines 3 et 4, puis comparées aux sorties des jobs.

5.3 MR1 — contrôle de cohérence

Pour chaque produit, la somme des fréquences `tf_raw` doit être égale au nombre total de tokens retenus de ses avis (contrôle du cahier des charges §8.1).

6. Lignes invalides (test T01)

Ligne	Défaut	Comportement attendu
13	JSON invalide : virgule manquante entre `title` et `text`, accolade fermante absente	Rejet, motif `json_invalide`
14	Champ `parent_asin` absent	Rejet, motif `parent_asin_absent`
15	`rating` = 7 (hors de [1, 5])	Rejet, motif `rating_hors_[1,5]`

Le pipeline doit **compter et signaler** ces rejets sans s'arrêter ni échouer silencieusement.

7. Vérification déjà réalisée

L'audit de schéma (`scripts/02_audit_schema.py`) exécuté sur ce fichier lit 15 lignes, retient 12 avis valides sur 3 produits et rejette exactement les 3 lignes ci-dessus, avec les motifs attendus. Les timestamps sont reconnus en millisecondes, de 2023-01-01 à 2023-04-01.

8. Limites

Ce jeu est volontairement minuscule et ne représente en rien la distribution du corpus réel (notes très déséquilibrées, avis longs, doublons, produits très évalués). Il sert uniquement à prouver l'**exactitude** des calculs ; la tenue sur des volumes réels est évaluée avec les lots L1 (1 %) et L2 (10 %).